

Machine Learning: Assignment 2 Report

Implementing Regression and Classification with Neural Networks on London Cars Dataset from 2014

Apardeep Singh Soni
500755463

School of Computer Science and
Engineering
Bangor University
Bangor, United Kingdom
prs25bgz@bangor.ac.uk

ABSTRACT

This report is divided into two parts, Part A contains answers of exercises from Lab 7, Lab 8 and Lab 10. While Part B contains the report presenting the development and evaluation of neural network models for predicting car prices using the LondonCars2014 dataset. The dataset originally contains 9,080 records across 11 features including make, model, year, mileage, engine type, and transmission. The work is structured across four tasks: exploratory data analysis (EDA), data preprocessing, regression neural network and classification neural network.

For regression, simple neural network without any validation and regularisation and neural network with regularisation and validation are built. Dropout and L2 regularisation along with K-Fold cross-validation using grid-search technique were employed to improve generalisation and avoid overfitting. For classification, car prices were binned into three categories (low, medium, high) and shallow neural network and deep neural network were evaluated. Comparison analysis with conventional ML models from assignment 1 was finally conducted to identify the effectiveness of neural networks.

Results achieved in this assignment were on local machine, Apple M4 with MPS GPU.

PART A

This section contains the answers to Lab 7, Lab 8 and Lab 10 exercises.

Lab07 - Exercise 0:

After implementing step 1 and step 2 the mean of obtained train and validation MSE's respectively are mentioned below for both linear and polynomial model of degree 9:

LINEAR models

train MSE: 0.0273

validation MSE: 0.0570

DEGREE-9 polynomial models

train MSE: 0.0000

validation MSE: 26300852.7671

- All linear models are very similar. The 15 lines cluster tightly with only small variations in slope and intercept. Linear models are stable across different training samples, i.e there is low variance.
- While degree-9 polynomial models are hugely different. The curves wiggle wildly and diverge sharply between data points and at the edges of the x-range. Each new split produces a noticeably different polynomial, i.e there is high variance.
- For the linear model, train and validation MSEs stay close and fairly stable. For the degree-9 polynomial, training MSE is near zero, but validation MSE is large and varies a lot from run to run.
- This is the bias-variance trade-off in action. The linear model is too rigid which means it has high bias and low variance, it remains consistent but underfits. While, degree-9 model is too flexible for only 10 training points, so it memorises them, including noise and fails to generalise, that's overfitting. The validation MSE reveals it, even though the training MSE is great.

All linear and degree-9 models can be seen in figure 1.

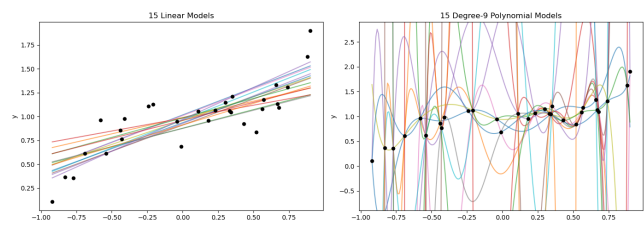


Figure 1. All linear models on left and degree-9 models are on right

Lab07 - Exercise 1:

- i. Train MSE: It Increases as λ increases ($0 \rightarrow 0.057$). The penalty forces weights to stay small, so the model can no longer fit every training point perfectly.
- ii. $w'w$: Decreases sharply as λ increases (approx. 9.6 billion $\rightarrow$ 1). The penalty term $\lambda w'w$ directly punishes large weights, shrinking them.
- iii. Test MSE: Follows a U-shape, starts huge approx. 6 million, severe which is an overfitting, and drops to a minimum at $\lambda=0.01$, then rises again at high λ because of underfitting. Small λ reduces variance, while too much λ introduces bias.

Lab07 - Exercise 2:

With 100 training points, test MSE for the degree-9 model is dramatically lower across all λ values. Even at $\lambda=0$ which means no regularisation, test MSE drops from 6 million (with 10 points) to just 0.0090. The U-shape across λ flattens, and the learned curve matches the true curve almost perfectly everywhere.

This is because with only 10 points, a degree-9 model had 10 parameters fitting 10 equations exactly, it memorised noise. With 100 points, it has 100 equations but still only 10 parameters, so it can no longer fit every point, forcing model to learn the true underlying pattern. More data itself acts as regularisation, it prevents overfitting without needing a penalty term.

Lab07 - Exercise 3:

The correct way is to use the validation set SSE to choose the best λ , because the validation set is held out from training and gives an unbiased estimate of generalisation. Using training error would always favour $\lambda=0$ (overfitting), and using test error would leak the test set into the model selection process, making the final test error meaningless.

train_MSE: 0.9674883263173008

test_MSE: 0.1946372967441878.

Lab07 - Exercise 4:

Using the cumulative plot data, 56 principal components are needed to reach 99% reconstruction accuracy. This matches the value from Q1, where I tested incrementally and found that nPCA=55 gives a reconstruction error of 0.0101 (just above 1%), while nPCA=56 gives 0.0098 (just below 1%).

Both methods agree because they measure the same thing from different angles. Eigenvalues describe how much variance each PC captures, and reconstruction error is the variance the PCs fail to capture. When the cumulative eigenvalue sum reaches 99%, the missing 1% corresponds directly to the reconstruction error. The cumulative method is more efficient, it gives the exact threshold without needing to rebuild any images, while the Q1 method confirms the result by direct measurement.

Lab07 - Exercise 5:

The maximum face recognition accuracy was 70.7% at nPCA = 27 among (1, 2, 5, 10, 15, 20, 25, 27, 30, 40, 50, 75, 100, 150), which is better than the raw image accuracy of 67.1%. PCA improves accuracy because it filters out noise. The first principal components capture the meaningful variation between faces, while the discarded ones mostly contain pixel-level noise. With too few PCs, important information is lost and accuracy drops to 20–58%, and with too many, noisy components are reintroduced, and accuracy falls back to the raw-image level. The sweet spot at 27 PCs gives KNN a cleaner, lower-dimensional space where distances between faces are more meaningful.

Lab08 - Exercise 0:

The completed cell code is:

```
Yhat_test = hit_lm.predict(X_test)
lm_train_mae = MeanAbsoluteError()
mae_value =
lm_train_mae(torch.tensor(Yhat_test),
torch.tensor(Y_test))
```

```
print(f"The MAE of the test dataset
for the Least Square model is
{mae_value:.2f}")
```

The MAE of the test dataset for the Least Square model is 259.72

Lab08 - Exercise 1:

Cross-validation chose the best value of λ as:

$$\lambda=176.2148325524328$$

Lab08 - Exercise 2:

The features that have been eliminated from the model are the ones with coefficients equal to 0.

The eliminated features are:

- HmRun
- Runs
- Years
- League
- Division
- Errors
- NewLeague

This conclusion is reached because in regression models such as Lasso regression, a coefficient of exactly 0.00 means that the feature does not contribute to predicting the target variable and has therefore been removed/eliminated from the model.

Lab08 - Exercise 3:

In this only 1 hidden layer is defined

Activities in the hidden layer:

- Linear Transformation: The 19 input features are mapped to 50 neurons using learnable weights and biases: $z = W_1x + b_1$
- ReLU Activation: Applied to each of the 50 neurons individually. Any negative value becomes zero: $h = \max(0, z)$
- Dropout Regularisation: 40% of the 50 neurons are randomly switched off during each training iteration to prevent overfitting.

Lab08 - Exercise 4:

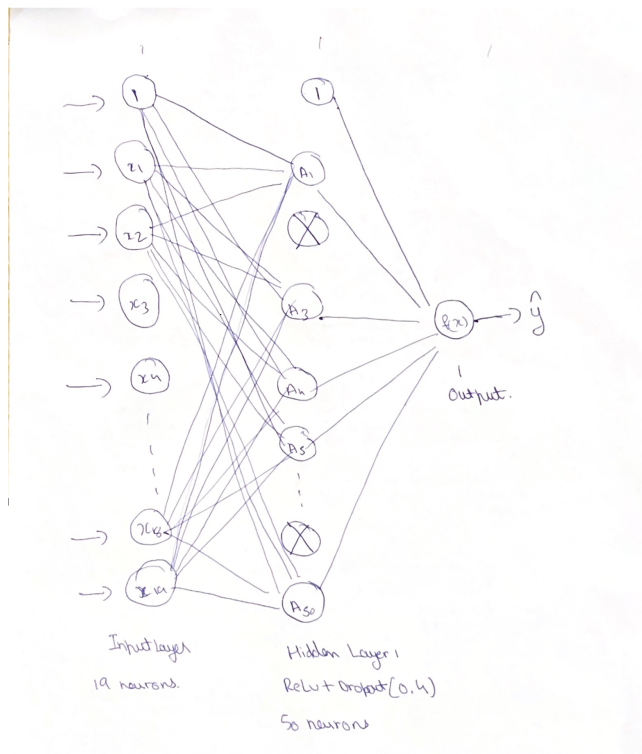
Input layer has 0 parameters. It just holds the raw data, no weights or biases.

Hidden layer `nn.Linear(19, 50)`:

- Weights: $19 \times 50 = 950$
- Biases: 50
- Total = 1000

Output layer `nn.Linear(50, 1)`:

- Weights: $50 \times 1 = 50$
- Biases: 1
- Total = 51



Lab10 - Exercise 0:

Switching the validation rate from 0.2 to 0.8 flips the split, only 20% of the data would be used for training and 80% would be set aside for validation. The model would likely perform worse overall. With so little training data, it wouldn't have enough examples to properly learn what each digit looks like, so training accuracy would probably drop, and the model might be overfit to the small training set. On the other side, the validation results would be more stable since they'd be measured on a bigger chunk of data, but that doesn't help much when the model itself is undertrained. So, 0.2 is the better choice here. It gives the model enough data to actually learn from while still keeping a reasonable validation set to track progress.

Lab10 - Exercise 1:

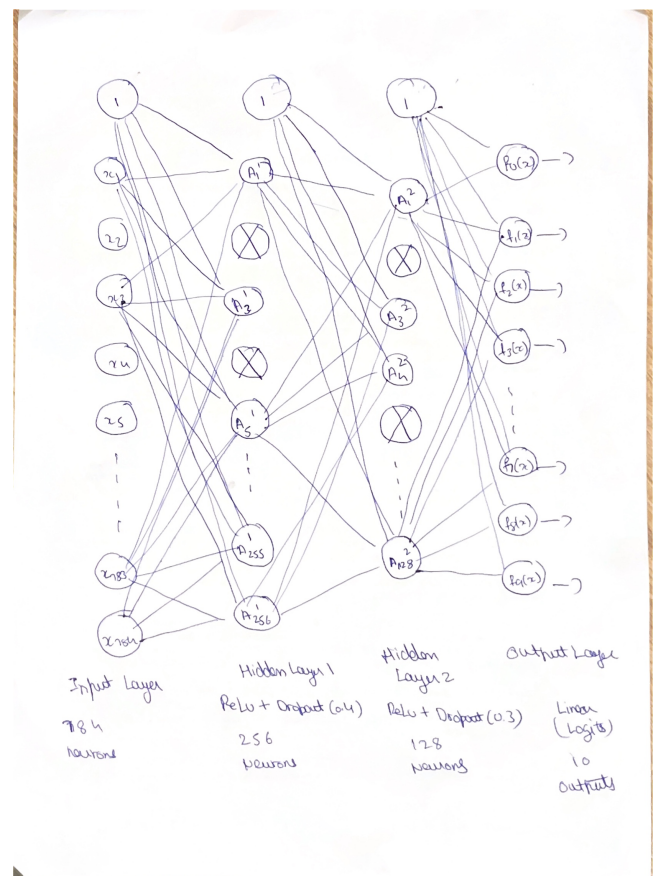
Number of parameters need to be trained in this neural network would be:

$$W_1: \text{Linear}(784, 256) - (784 \times 256) + 256 = 200,960$$

$$W_2: \text{Linear}(256, 128) - (256 \times 128) + 128 = 32,896$$

$$W_3: \text{Linear}(128, 10) - (128 \times 10) + 10 = 1,290$$

$$\text{Total } 235,146$$



Lab10 - Exercise 2:

Yes, Dropout is the regularisation technique used in this architecture. It appears in two places, Dropout(0.4) in Layer 1 and Dropout(0.3) in Layer 2.

Function of each layer:

- Flatten: reshapes the 28×28 input image into a flat 784-value vector so it can feed into a Linear layer. No learnable parameters.
- Linear(784 to 256): first fully connected layer. Learns 256 new features from the raw pixel values.
- ReLU: introduces non-linearity by replacing negatives with zero. Without it, stacked linear layers would collapse into a single linear transformation.
- Dropout(0.4): randomly disables 40% of neurons during training, forcing the network to spread learning across neurons rather than depending on a few. Prevents overfitting.
- Linear(256 to 128): second fully connected layer, compressing into 128 more abstract features.
- ReLU + Dropout(0.3): same purpose as before, lower dropout rate suits the narrower layer.
- Linear(128 to 10): output layer producing one score per digit (0–9). The class with the highest score is the prediction.

Lab10 - Exercise 3:

One epoch corresponds to 188 gradient steps. This can be seen directly from the training progress bar which shows 188/188. The original training set has 60,000 images, but 20% was set aside for validation, so only 48,000 images are actually used for training. With a batch size of 256, that gives $48,000/256 = 187.5$, which rounds up to 188 batches. Since weights are updated once per batch, one epoch = 188 gradient steps.

Lab10 - Exercise 4:

The four mini-batch sizes gave test accuracies of 95.47% (size 50), 96.09% (size 100), 96.19% (size 200), and 96.52% (size 400). Test loss increased monotonically with batch size: 0.446 → 0.589 → 0.771 → 1.031.

Two clear trends emerge as the mini-batch size increases:

- Training time decreases sharply (21 → 13 → 8 → 6 minutes). Larger batches mean fewer gradient updates per epoch, and each update uses more samples in parallel, so the per-epoch wall-clock time drops. This is the main practical reason to use larger batches when hardware allows.
- Test accuracy slightly improves, but test loss also increases, meaning the model gets more answers right but is less confident in them. Smaller batches produce noisier gradient updates that act like implicit regularisation, giving sharper predictions; larger batches give smoother updates with faster training but less calibrated confidence.

PART B

DATA PREPROCESSING

Performed EDA

- The dataset has 9,080 rows and 11 columns, with no missing values.
- On further exploring, out 11 features, 7 are categorical features with 'object' Dtype and and 4 are numeric with int64 Dtype. This makes dataset to consume 780.4+ KB memory.
- To make efficient use of memory, Dtype of categorical variables are transformed from 'object' to 'category', and Dtype of three numerical variables are transformed from int64 to int32, while Dtype for Price kept same. This reduces the memory consumption to 292.3 KB.
- Analysed Correlation between Price and Mileage, which showed a clear negative correlation between price and mileage, as mileage increases, price decreases, can be seen in figure 2.

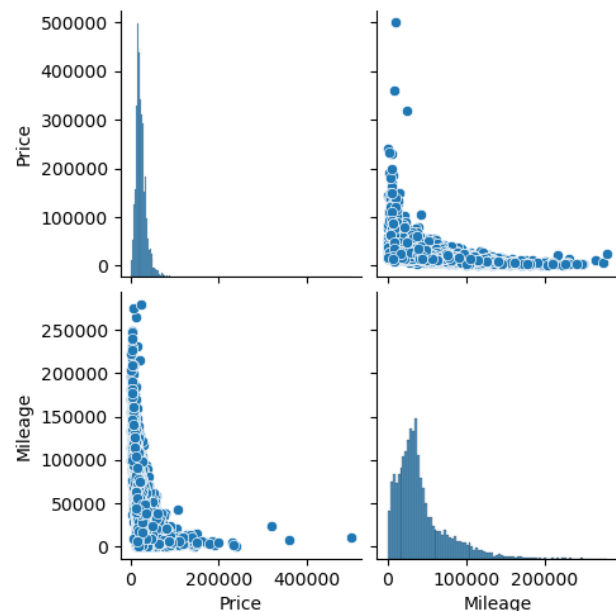


Figure 2. Illustrating correlation between price and mileage variables.

Encoding Categorical Variables

Since the dataset contains 9,080 samples and 11 features, of which 7 are categorical and require encoding as neural networks need everything to be numeric. The three numeric columns (Year, Mileage, and Doors) need no encoding, while Price is the target variable.

For encoding a two different encoding strategy was adopted depending on cardinality.

One-Hot Encoding (OHE) was used for all low-to-medium cardinality nominal columns because it avoids implying any false ordinal relationship between categories. This is crucial for problems like regression, where numeric values are treated as having mathematical magnitude, and assigning arbitrary integers to categories would mislead the model into assuming an ordering that does not exist. The "drop_first=True" argument was applied to avoid the dummy variable trap, basically, if you keep all the one-hot columns, they sum to 1 for every row, which creates perfect multicollinearity. Dropping one column per group removes that redundancy.

The only exception was the Model column, which contains 1,060 unique values. This one is tricky because there are 1,060 unique car models. One-hot encoding that column would have added 1,060 sparse columns to the feature matrix, which would have eaten memory and probably caused overfitting. Label encoding compresses everything into a single integer column, at the cost of imposing a meaningless numeric order. It's not ideal for a neural network, but for this kind of cardinality it's the practical choice.

After encoding, the feature matrix went from 11 columns to 93.

Discovering duplicate rows

This was something I noticed late in the project. When I checked `df.duplicated().sum()`, it came back with 922 duplicates, about 10% of the dataset, which left us with 8,158 rows in the dataset.

Duplicates are a problem because of train/test leakage. If the same car ends up in both the training set and the test set, the model basically memorises it during training and gets a free correct answer at test time. That makes the test performance look better than it really is.

Later in the report the comparisons between both attempts before and after deduplication has been shown, and the differences are honestly more interesting than I expected.

Train/Test Split

Then the dataset was split into training and testing sets using a 70/30 ratio with `random_state=42` for reproducibility, which can be seen in table 1.

Split	Samples	Percentage (%)
Training set	5,710	70
Testing set	2,448	30
Total	8,158	100

Table 1. showing train/test split of the dataset

For the classification task, `stratify=y` has also been used because the classes are heavily imbalanced, 70% of cars fall in the "medium" band, only 4.6% are "high". Stratification keeps those proportions consistent across train and test.

Normalisation

Normalisation is the process of rescaling of numerical features to a standard range or distribution. All features were standardised with `StandardScaler`, fitted exclusively on the training set to avoid data leakage and then applied to both sets.

One thing worth being explicit about, one-hot encoded columns are scaled too. After scaling, a 0 in a one-hot column becomes something like -0.4 , and a 1 becomes around $+2.4$. The binary information is preserved (the column still has only two unique values), but everything is on a similar numeric range. For a neural network this matters because gradient descent is more stable when all inputs occupy comparable scales, otherwise, larger-valued features dominate gradient updates regardless of their actual importance.

For regression, the target Price is also scaled. This stops the loss values from being enormous, which would otherwise destabilise gradients. For classification, the target is categorical and doesn't get scaled.

REGRESSION

The regression network follows the architecture:

Input (93 features) $\rightarrow$ Dense(50, ReLU) $\rightarrow$ Dense(20, ReLU) $\rightarrow$ Dense(1)

This gives 5,741 trainable parameters in total: 4,700 in the first hidden layer (93×50 weights + 50 biases), 1,020 in the second ($50 \times 20 + 20$), and 21 in the output layer ($20 \times 1 + 1$).

Two versions of this model were trained. The baseline version (no regularisation, no validation), and the improved version with dropout, L2 regularisation, and 5-fold cross-validation for hyperparameter selection.

Common training settings for both versions of the model:

- Loss function: Mean Squared Error (MSE)
- Learning rate: 0.001
- Batch size: 100
- Epochs: 30
- 70/30 train/test split with `random_state=42`

Regression Neural Network without optimisation

The baseline model trains the architecture exactly as described above, with no regularisation and no validation set. It serves as a reference point, it shows what the bare architecture achieves before any hyperparameter tuning or regularisation is added.

Hyperparameters used:

Hidden Layers: Two hidden layers are chosen for this as per assignment requirements, which allows the network to learn non-linear combinations of features that a linear model cannot capture. A single hidden layer would be sufficient as a universal

approximator in theory, but two layers tend to learn more efficiently in practice.

neuron counts (50, then 20): number of neurons kept same as per assignment requirements, where wider first layer provides capacity to capture varied feature interactions, while a narrower second layer compresses these into a more focused representation before the output.

ReLU activation: was chosen as per assignment requirements, it also avoids the vanishing gradient problem that affects sigmoid and tanh in deeper networks. It is also computationally cheap, simply $\max(0, x)$, speeding up both forward and backward passes.

MSE loss: is the standard regression loss. It penalises large errors quadratically, making the model sensitive to outliers. Combined with the standardised target, MSE produces well-behaved gradients during training.

Batch size 100: balances two competing concerns, smaller batches give noisier gradient updates that can help escape local minima but train more slowly per epoch, while larger batches give smoother updates and faster training but may converge to less generalisable solutions.

Thirty epochs: The training loss curve (can be seen in figure 3) confirms convergence happens well before the 30-epoch budget, by around epoch 5 the loss has plateaued.

Output:

The training loss curve drops sharply in the first few epochs and plateaus around 0.06 by epoch 5, with mild fluctuation thereafter, clear evidence of convergence well within the training budget.

Metric	Test value
MSE	0.264
R ²	0.713

Table 2. Showing MSE and R2 metrics of baseline model

An R² of 0.713 means the model captures about 71% of the variability in test-set prices. The remaining 29% is residual noise the network can't capture from these features.

However, before deduplication, the baseline produced MSE = 0.114 and R² = 0.864. These inflated numbers reflected train/test leakage.

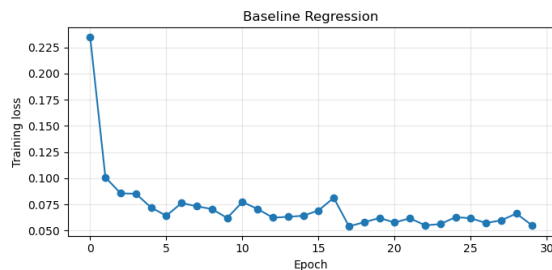


Figure 3. Train loss curve for baseline model

Improved Regression Neural Network (with regularisation and cross-validation)

The improved version uses the same architecture but adds two regularisation techniques and proper hyperparameter selection via cross-validation.

Hyperparameters and techniques:

The architecture, loss function, batch size, and epochs are kept identical to the baseline so any change in performance is attributable to the new additions only.

RMSprop :was used as the optimiser. It's an adaptive learning-rate method and the default in the ISLP SimpleModule helper, matching the convention from the labs.

Learning rate: learning rate of 0.001 is a widely-used conservative default. PyTorch's default for RMSprop is 0.01, but 0.001 is less prone to divergence on small or sensitive networks.

Dropout: was added after each hidden layer. During training, dropout randomly deactivates a percentage of neurons in a layer, which prevents the network from depending too heavily on any single neuron or pathway. This forces learning to be distributed across multiple neurons and improves generalisation.

L2 regularisation: was applied through the optimiser's weight_decay parameter. This adds a $\lambda \times \sum w^2$ penalty term to the gradient update, keeping the weights small. Smaller weights produce smoother decision boundaries that are less likely to overfit training noise.

5-fold cross-validation grid search: was used to select the dropout rate and λ . The search space:

- Dropout values: {0.2, 0.3, 0.4}
- λ values: {0.001, 0.01, 0.1, 1, 10, 50, 100, 500, 1000}

That's 27 combinations $\times$ 5 folds = 135 training runs. The λ range was deliberately wide to observe the breakdown point of regularisation. Inside each fold loop, seed_everything(0) was called before each model so every configuration started from identical random conditions, this ensures observed differences came from hyperparameters and not random initialisation noise.

Grid search was preferred over Bayesian optimisation because the search space is small (27 combinations), making grid search exhaustive and easy to interpret in a table.

CV findings:

The best combination was dropout = 0.2, λ = 0.001, with mean fold validation MSE of 0.196058.

For $\lambda \geq 1$, the model collapsed entirely (validation MSE $\approx$ 1.0, equivalent to predicting the mean every time). The L2 penalty was so strong that all weights were forced toward zero, making the network unable to learn anything. This shows where regularisation strength becomes destructive.

Lower dropout (0.2) consistently outperformed higher dropout (0.4) across the grid. This suggests the model wasn't overfitting heavily to begin with, the network is small relative to the data, so there was little for dropout to "fix."

Output:

The training loss curve (can be seen in figure 4) for the improved model shows a higher final value (~0.13) compared to the baseline (~0.06). At first glance this looks worse, but the test MSE is actually lower. The dropout forces the model to work harder during training (neurons are randomly switched off), so training loss stays higher, but the resulting model generalises better.

Metric	Test value
MSE	0.136
R ²	0.852

Table 3. Showing MSE and R² metrics of baseline model

However, before deduplication, the improved model produced MSE = 0.107 and R² = 0.872. The gap between original and deduplicated results is much smaller here than for the baseline.

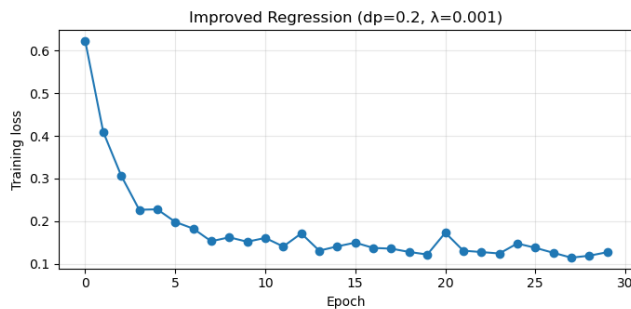


Figure 4. Train loss curve for improved model

Comparison of the two regression models

Model	Test MSE	Test R ²
Baseline	0.264	0.713
Improved (dropout=0.2, λ=0.001)	0.136	0.852

Table 4. Comparison between baseline and improved model

The improvement is substantial as can be seen in table 4, MSE roughly halves and R² jumps by nearly 14 percentage points. This is much bigger than the small numerical gap seen on the pre-deduplication data (which made it look like regularisation only helped a little). The deduplicated comparison makes a much stronger case for regularisation, without it, the network struggles to generalise from a clean training set.

The training loss curves also tell different stories. The baseline has a much lower training loss (~0.06) than its test MSE (0.264), indicating heavy overfitting on the cleaned data. The improved model's training loss (~0.13) is similar to its test MSE (0.136), the

train-test gap has nearly closed, which is exactly what regularisation is designed to achieve.

Comparison with conventional methods (from Assignment 01)

In Assignment 01, a Random Forest regressor on the full feature set produced a train MSE of 0.019 and a test MSE of 0.224 (on the original duplicate-containing data), which was very similar on given three features. This large gap between train and test indicates the RF was overfitting, it memorised the training data but generalised less well to the test set.

A direct, fair comparison would require rerunning the A1 random forest on the same deduplicated dataset used for the neural networks. Based on the A1 numbers:

Model	Test MSE
Random Forest Regressor (A1, original data)	0.224
Neural Network — baseline (deduplicated)	0.264
Neural Network — improved (deduplicated)	0.136

Table 5. Showing Test MSE values for neural network model and random forest model

The improved neural network gives the lowest test MSE. The baseline NN actually performs worse than the A1 RF on this metric, which is consistent with the broader finding that regularisation is essential here. Without it, the network underperforms a much simpler conventional model.

CLASSIFICATION

For the classification task, prices were binned into three classes:

- low: price < £15,987
- medium: £15,987 ≤ price ≤ £48,500
- high: price > £48,500

Class distribution: 25.0% low, 70.5% medium, 4.6% high.

The labels were mapped to {low: 0, medium: 1, high: 2}, preserving the natural price order. This makes confusion matrices much easier to read, low and high appear at opposite ends of the matrix rather than next to each other (which would happen with alphabetical encoding).

The heavy imbalance is important context for evaluation. A model that always predicts "medium" would score ~70% accuracy without learning anything useful, so accuracy alone is misleading. Precision, recall, and F, especially macro-averaged, give a much more honest picture, particularly for the minority "high" class.

Note on hyperparameter selection for classification:

The original task specification called for the Swish activation function with L2 regularisation (λ = 0.01) for the shallow network,

and Swish activation with 40% dropout for the deep network. Rather than using these fixed values directly, 5-fold cross-validation was applied to both networks to find the best combination of activation function and regularisation strength. The specified values appear naturally as one row in each CV table, and Sections 3.1 and 3.2 compare the CV winners against these defaults.

Shallow Neural Network

The architecture:

Input (93) → Dense(150, Swish) → Dense(3)

Training settings: cross-entropy loss, L2 regularisation, batch size 100, 30 epochs, 70/30 stratified split, RMSprop optimiser with learning rate 0.001.

A 5-fold cross-validation grid search was used to select the activation function and L2 regularisation strength. The grid was:

- Activations: {ReLU, LeakyReLU, Sigmoid, Tanh, Swish}
- λ values: {0.001, 0.01, 0.1}

That's $5 \times 3 \times 5 = 75$ training runs.

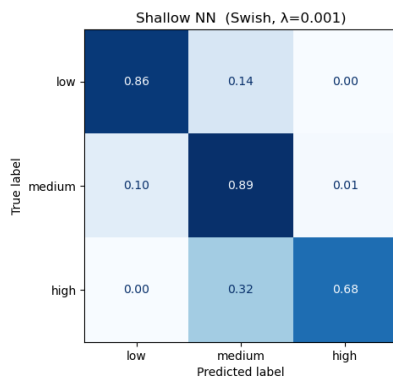
Top CV results:

Swish	0.001	0.893345
ReLU	0.001	0.892469
LeakyReLU	0.001	0.892469
ReLU	0.010	0.891944
Sigmoid	0.100	0.736252

The top three activations at $\lambda=0.001$. L2 strength has a much larger effect: $\lambda=0.1$ drops accuracy by 4–5 points regardless of activation. Sigmoid at $\lambda=0.1$ collapses to 72.7%, which shows vanishing gradient problem in action, with sigmoid's already-small gradients being killed off by the heavy L2 penalty.

The default specification (Swish + $\lambda=0.01$) gave 88.4% — close to optimal but not the best. Based on the CV result, Swish + $\lambda=0.001$ was chosen for the final shallow model.

Test results: Confusion matrix for shallow neural network can be seen in figure 5 below.



Metric	Value
Accuracy	0.8734
Precision (macro)	0.8281
Recall (macro)	0.8100
F1 (macro)	0.8163

Table 6. Showing the quality matrices for shallow neural network

However, before deduplication, the shallow NN gave accuracy 0.9023 and macro F1 0.8455

Per-class breakdown for shallow neural network:

Class	Precision	Recall	F1 Score
low	0.76	0.86	0.80
medium	0.93	0.89	0.91
high	0.80	0.68	0.74

Table 7. showing per class break down

Deep Neural Network

The architecture:

Input (93) → Dense(162) → Dense(81) → Dense(27) → Dense(9) → Dense(3) (each hidden block: Linear → activation → Dropout)

This gives 30,927 trainable parameters, roughly $5\times$ the shallow network's 14,403.

Training settings: cross-entropy loss, dropout regularisation, batch size 100, 30 epochs, 70/30 stratified split.

A 5-fold CV grid was run over:

- Activations: {ReLU, LeakyReLU, Sigmoid, Tanh, Swish}
- Dropout rates: {0.2, 0.3, 0.4, 0.5}

That's $5 \times 4 \times 5 = 100$ training runs.

Top CV results:

ReLU	0.3	0.892294
LeakyReLU	0.4	0.891419
Swish	0.3	0.890543
Swish	0.4	0.889492
LeakyReLU	0.2	0.889492

This follows same pattern as the shallow CV, top combinations tightly clustered with very small differences. The default (Swish + dropout=0.4) gave 88.9%, close to but slightly below the CV winner.

Dropout of 0.5 hurt performance across all activations, suggesting 50% is too aggressive for this architecture.

Final deep nn model was trained with: ReLU + dropout=0.3

Test results: Confusion matrix for deep neural network can be seen in figure 6 below.

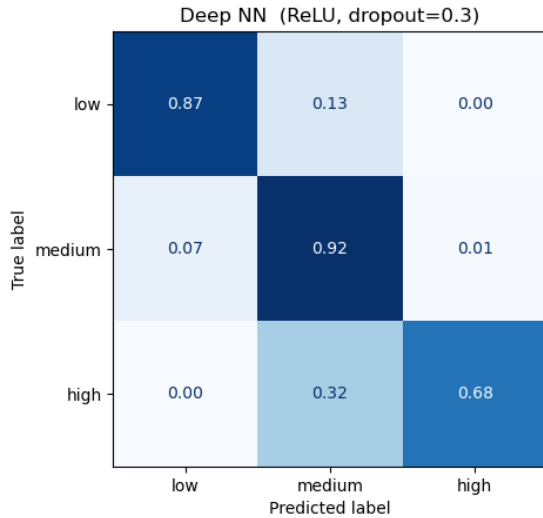


Figure 6. showing confusion matrix for deep neural network

Metric	Value
Accuracy	0.8954
Precision (macro)	0.8594
Recall (macro)	0.8222
F1 (macro)	0.8379

Table 8. Showing the quality matrices for deep neural network

Per-class breakdown for deep nn model:

Class	Precision	Recall	F1 Score
low	0.81	0.87	0.84
medium	0.93	0.92	0.93
high	0.84	0.68	0.75

Table 9. showing per class break down

However, before deduplication, the deep NN gave accuracy 0.9023 and macro F1 0.8393

A small but important implementation detail

The final layer of both classifiers is a plain nn.Linear, with no softmax. This is correct for PyTorch, nn.CrossEntropyLoss already applies softmax internally as part of a numerically-stable combined operation. Adding an explicit softmax layer would result in double application, which makes the gradients tiny and breaks training.

Macro vs weighted averaging

Macro averaging was used for precision, recall, and F1. Weighted averaging would weight each class by its size, so the majority

"medium" class would dominate the average and hide poor performance on the minority "high" class. Macro takes the simple unweighted average across classes, treating them all equally. For imbalanced data, macro is the more honest metric.

Comparison of the two classifiers

Comparison of both shallow and deep neural network can be seen in table 10.

Metric	Shallow NN	Deep NN
Accuracy	0.8734	0.8954
Precision (macro)	0.8281	0.8594
Recall (macro)	0.8100	0.8222
F1 (macro)	0.8163	0.8379
Parameters	14,403	30,927

Table 10. Showing comparison between shallow and deep neural network

The deep network has slightly better accuracy and F1, but the shallow network has slightly better recall. The differences are small, and both models cluster around the same overall performance level despite the deep network having more than twice as many parameters.

A consistent pattern across both classifiers: the "high" class has the lowest recall (0.72 shallow, 0.68 deep), confirming that the model under-predicts the minority class regardless of architecture.

Comparison with conventional methods (from Assignment 01)

In Assignment 01, two conventional classifiers were tuned on the same 3-class binning problem, Logistic Regression (with L1 regularisation and a Random Forest classifier with tuned hyperparameters. Both used the original duplicate-containing data.

Quality matrices comparison across models from assignment 1 and neural networks can be seen in table 11.

Model	Accuracy	Macro F1	Macro Recall
Logistic Regression (A1, tuned)	0.8245	0.62	0.57
Random Forest (A1, tuned)	0.8216	0.66	0.62
Shallow NN (A2, deduplicated)	0.8734	0.8163	0.8100
Deep NN (A2, deduplicated)	0.8954	0.8379	0.8222

Table 11. showing comparison between conventional and neural network models for classification problem

Both neural networks outperform the conventional methods on every metric, with the largest gap appearing in macro F1 and macro recall. This is driven primarily by the minority "high" class, where the conventional models struggle severely

Model	"high" Precision	"high" Recall	"high" F1
Logistic Regression (A1)	0.79	0.18	0.29
Random Forest (A)	0.50	0.33	0.40
Shallow NN (A2)	0.80	0.68	0.74
Deep NN (A2)	0.84	0.68	0.75

Table 12. showing predictions about minority class by different models.

The A1 Logistic Regression catches only 18% of high-priced cars (recall = 0.18), it predicts "high" only when it's very confident, which means it misses the vast majority. The A1 Random Forest catches a third (recall = 0.33). The neural networks catch roughly 70% of them (recall $\approx$ 0.68–0.72), more than double either conventional method catches a third (recall = 0.33). The neural networks catch roughly 70% of them (recall $\approx$ 0.68–0.72), more than double either conventional method.

The same pattern appears in the "low" class recall too: conventional methods catch about 60%, the neural networks catch 87–89%. So the neural networks aren't just slightly better, they handle the imbalanced classes substantially more effectively, even without explicit class-weighting techniques.

CONCLUSION

Several findings stand out from this work:

1. Neural networks performed competitively on both tasks. On the cleaner deduplicated data, $R^2 \approx 0.85$ for regression and accuracy ≈ 0.87 –0.90 for classification, matching or exceeding the conventional methods from Assignment 01.
2. Depth gave only marginal benefits. The deep classifier with 30,927 parameters performed only slightly better than the shallow one with 14,403. For tabular data of this size, simpler architectures are nearly as effective.
3. Regularisation is essential to get optimal results. The deduplication experiment revealed that without regularisation, the regression baseline lost 0.15 in R^2 when leakage was removed, the regularised model lost only 0.02. This is a much stronger case for regularisation than the marginal improvement visible on the duplicate-containing data suggested.
4. Class imbalance is the persistent bottleneck. Across all classification models, the "high" class (4.6% of data) is the weakest performer. None of the architectural or hyperparameter choices addressed this, it's a data-level problem that requires data-level solutions.